

Proyecto Final

BUBBLE

Documento de Arquitectura

REALIZADO POR:

Rocío Martinez

Martín Murciano

Juan Pavlavicius

Índice

Arquitectura	3
Objetivo del documento	3
Objetivos de la Arquitectura	3
Estilo de Arquitectura seleccionado	3
Descripción de las Capas	3
Capa de Presentación	3
Capa de Lógica de Negocio	4
Capa de Persistencia de Datos	4
Uso de Microservicios	4
Microservicios Implementados:	4
Consideraciones de Seguridad	5
Capa de Presentación:	5
Capa de Lógica de Negocio:	5
Capa de Persistencia de Datos:	5
Escalabilidad y Rendimiento	5
Herramientas y Tecnologías	5
Plan de Despliegue	5
Conclusión	5

Arquitectura

Objetivo del documento

El sistema de Bubble está diseñado para ser una plataforma centralizada que permite a los organizadores gestionar eventos, a los clientes adquirir entradas y a los administradores supervisar las operaciones. La arquitectura elegida debe cumplir con criterios de escalabilidad, modularidad, mantenibilidad y seguridad.

Objetivos de la Arquitectura

- Garantizar un diseño modular que permita agregar nuevas funcionalidades sin afectar las existentes.
- Asegurar la alta disponibilidad y el buen rendimiento bajo cargas de tráfico elevadas.
- Proporcionar una experiencia de usuario intuitiva y eficiente tanto en la versión web como móvil.
- Cumplir con estándares de seguridad para proteger datos sensibles, como información personal y bancaria.
- Facilitar la interoperabilidad entre los módulos del sistema, como compra de entradas, creación de eventos, y validación de organizadores.

Estilo de Arquitectura seleccionado

Para este proyecto, se ha adoptado una arquitectura en capas complementada con principios de microservicios para funcionalidades clave. Esto combina la simplicidad de las capas con la flexibilidad y escalabilidad de los microservicios.

- **Arquitectura en capas:** Divide el sistema en niveles jerárquicos (presentación, lógica de negocio, datos), lo que facilita la separación de responsabilidades y la mantenibilidad.
- **Microservicios:** Los módulos principales, como el procesamiento de pagos y la gestión de calificaciones, están desacoplados para permitir su escalabilidad independiente.

Descripción de las Capas

Capa de Presentación

- **Responsabilidad:** Interactuar directamente con los usuarios finales.
- **Componentes:**
 - Interfaz de usuario web: Desarrollada en React.js, optimizada para accesibilidad y rendimiento.
 - Aplicación móvil: Implementada en React Native para soporte multiplataforma.
- **Características:**
 - Interfaces intuitivas para clientes, organizadores y administradores.
 - Adaptabilidad a dispositivos móviles (responsive design).

Capa de Lógica de Negocio

- **Responsabilidad:** Gestionar las reglas de negocio y coordinar las operaciones entre la presentación y los datos.
- **Componentes:**
 - Módulo de Compra de Entradas.
 - Módulo de Creación de Eventos.
 - Módulo de Calificación de Eventos.
 - Módulo de Administración (gestión de usuarios)
- **Características:**
 - Implementación de API REST para facilitar la comunicación entre las capas.

Capa de Persistencia de Datos

- **Responsabilidad:** Almacenar y recuperar datos de forma segura y eficiente.
- **Componentes:**
 - Base de datos relacional: MySQL, estructurada para manejar transacciones y consultas complejas.
- **Características:**
 - Replicación y particionamiento para mejorar la disponibilidad.
 - Cifrado de datos sensibles en reposo y en tránsito.

Uso de Microservicios

Los microservicios son utilizados para funciones específicas que requieren alta escalabilidad o independencia operativa.

Microservicios Implementados:

- Servicio de Procesamiento de Pagos:
 - Integración con plataformas de pago como Mercadopago.
 - Garantiza transacciones seguras mediante estándares PCI DSS.
- Servicio de Notificaciones:
 - Gestiona notificaciones push y correos electrónicos para recordatorios de eventos o confirmaciones de compra.
- Servicio de Calificaciones:
 - Procesa y almacena las calificaciones de eventos, con cálculos en tiempo real de popularidad.
- Ventajas de Microservicios en este Sistema:
 - Escalabilidad: Cada servicio puede ampliarse independientemente según la demanda.
 - Resiliencia: Fallos en un servicio no afectan al resto del sistema.

Consideraciones de Seguridad

Capa de Presentación:

- Uso de HTTPS para todas las comunicaciones.
- Autenticación multifactor (MFA) para organizadores y clientes.

Capa de Lógica de Negocio:

- Validación estricta de entradas para prevenir inyecciones SQL y XSS.
- Tokenización de sesiones para evitar secuestro de sesiones.

Capa de Persistencia de Datos:

- Cifrado de contraseñas con algoritmos como bcrypt.

Escalabilidad y Rendimiento

- **Balanceo de carga:** Implementación de balanceadores para distribuir las solicitudes entre servidores.
- **Caching:** Uso de Redis o Memcached para almacenar datos frecuentemente accedidos, como detalles de eventos populares.
- **Autoescalado:** Configuración de autoescalado en plataformas en la nube.

Herramientas y Tecnologías

- **Frontend:** React.js (Web), React Native (Móvil).
- **Backend:** Node.js con Express.
- **Base de Datos:** MySQL.
- **Contenedores:** Docker para empaquetar servicios.

Plan de Despliegue

- **Entorno de desarrollo:** Entornos locales con Docker Compose.
- **Entorno de pruebas:** Servidores en la nube para pruebas funcionales y de integración.
- **Entorno de producción:** Configuración de infraestructura escalable en la nube con alta disponibilidad.

Conclusión

La arquitectura del sistema equilibra la simplicidad de una arquitectura en capas con la flexibilidad de microservicios para garantizar escalabilidad, rendimiento y facilidad de mantenimiento. Este diseño modular permite responder a futuras necesidades del negocio y a un crecimiento sostenido del sistema.